Report BRDB James Zeiger

SETUP

Init

```
docker compose -f docker-compose.yaml -f postgres-docker-compose
yaml -f redis-docker-compose.yaml up -d
```

Run

You can easily run each test by choosing your amount of records to insert and test on (n=...) and run the python file.

Stop

```
docker stop post|mango|red
```

PROJECT

Details

To start, we choose a number of n records we are going to insert into our databases.
We then time every single operation with python's time library: time.perf_counter(), as it has a high precision.
Then the connection to the database is established and is wiped for clean testing.

The creation operation is inserting one and n records, each containing {x_pos, y_pos, user, score, label}, where:

- x_pos is a random integer between 1 and 100
- y_pos is a random integer between 1 and 100
- user is unique to each record ranging between 1 and n
- score is a random number between 0 and 1000
- label is a string picked at random with {alpha, beta, gamma}

Each database has its own syntax for creation and insertion.

For reading, we test read all, read one without index, read one with index and one by label as there is a difference between numbers and strings search. (Tests connection speed)

To test the updating, we modify one and many (~one third of n). (Tests search and calculation speed)

And finally the delete operation, where we delete one record, many and the entire database content. (Test formatting speed)

Results

```
PostgreSQL (1,000 records)

--- CREATE ---
Insert one record: 3.34 ms
Insert many 1,000: 6.68 ms

--- READ ---

Select all 1,001 records: 1.29 ms
Find user 42 (no index): 1.77 ms  -> 1 row(s)
Find user 42 (with index): 0.77 ms  -> 1 row(s)
Find label 'alpha': 0.74 ms  -> 333 row(s)

--- UPDATE ---

Update one user=42: 1.26 ms
Update many label=beta: 3.13 ms  -> 357 row(s)

--- DELETE ---

Delete one user=1: 1.15 ms
Delete many label=gamma: 2.04 ms  -> 310 row(s)
Drop table: 3.03 ms
--- --- FINISHED POSTGRESQL --- ---
```

```
PostgreSQL (10,000 records)

--- CREATE ---
Insert one record: 2.58 ms
Insert many 10,000: 36.65 ms

--- READ ---

Select all 10,001 records: 5.71 ms
Find user 42 (no index): 1.17 ms  -> 1 row(s)
Find user 42 (with index): 0.90 ms  -> 1 row(s)
Find label 'alpha': 2.24 ms  -> 3327 row(s)

--- UPDATE ---

Update one user=42: 1.25 ms
Update many label=beta: 24.69 ms  -> 3362 row(s)

--- DELETE ---

Delete one user=1: 1.52 ms
Delete many label=gamma: 3.72 ms  -> 3311 row(s)
Drop table: 2.22 ms
--- --- FINISHED POSTGRESQL --- ---
```

```
PostgreSQL (100,000 records)

--- CREATE ---
Insert one record: 2.67 ms
Insert many 100,000: 353.98 ms

--- READ ---

Select all 100,001 records: 37.20 ms
Find user 42 (no index): 3.16 ms  -> 1 row(s)
Find user 42 (with index): 0.57 ms  -> 1 row(s)
Find label 'alpha': 16.01 ms  -> 33342 row(s)

--- UPDATE ---

Update one user=42: 1.31 ms
Update many label=beta: 164.84 ms  -> 33290 row(s)

--- DELETE ---

Delete one user=1: 1.44 ms
Delete many label=gamma: 20.18 ms  -> 33369 row(s)
Drop table: 4.36 ms
--- --- FINISHED POSTGRESQL --- ---
```

```
MongoDB (1,000 records)

--- CREATE ---
Insert one record: 29.30 ms
Insert many 1,000: 6.31 ms

--- READ ---

Find all 1,001 records: 2.78 ms
Find user 42 (no index): 0.83 ms  -> 1 doc(s)
Find user 42 (with index): 1.64 ms  -> 1 doc(s)
Find label 'alpha': 2.36 ms  -> 339 doc(s)

--- UPDATE ---

Update one user=42: 1.09 ms
Update many label=beta: 5.47 ms  -> 325 doc(s)

--- DELETE ---

Delete one user=1: 0.69 ms
Delete many label=gamma: 4.05 ms  -> 336 doc(s)
Drop collection: 0.81 ms
--- --- FINISHED MONGODB --- ---
```

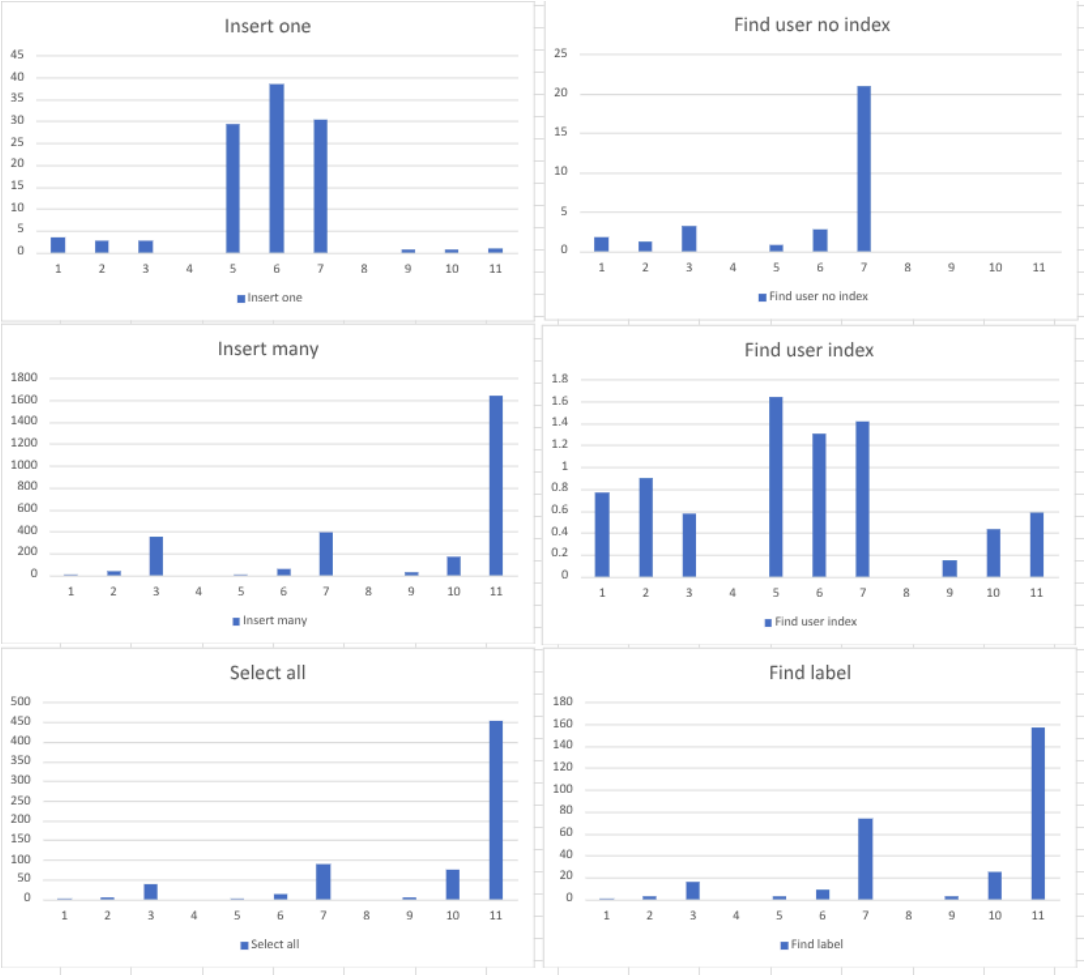
```
MongoDB (10,000 records)

--- CREATE ---
Insert one record: 38.42 ms
Insert many 10,000: 55.36 ms

--- READ ---

Find all 10,001 records: 12.14 ms
Find user 42 (no index): 2.78 ms  -> 1 doc(s)
Find user 42 (with index): 1.31 ms  -> 1 doc(s)
Find label 'alpha': 8.97 ms  -> 3336 doc(s)

--- UPDATE ---

Update one user=42: 0.72 ms
Update many label=beta: 19.96 ms  -> 3376 doc(s)

--- DELETE ---

Delete one user=1: 0.73 ms
Delete many label=gamma: 20.85 ms  -> 3289 doc(s)
Drop collection: 0.67 ms
--- --- FINISHED MONGODB --- ---
```

```
MongoDB (100,000 records)

--- CREATE ---
Insert one record: 30.46 ms
Insert many 100,000: 389.19 ms

--- READ ---

Find all 100,001 records: 89.75 ms
Find user 42 (no index): 21.04 ms  -> 1 doc(s)
Find user 42 (with index): 1.42 ms  -> 1 doc(s)
Find label 'alpha': 74.24 ms  -> 33414 doc(s)

--- UPDATE ---

Update one user=42: 0.76 ms
Update many label=beta: 198.69 ms  -> 33317 doc(s)

--- DELETE ---

Delete one user=1: 0.71 ms
Delete many label=gamma: 152.71 ms  -> 33270 doc(s)
Drop collection: 0.61 ms
--- --- FINISHED MONGODB --- ---
```

```
Redis Benchmark (1,000 records)

--- CREATE ---
Set 1 record: 0.73 ms

Pipeline set 1,000 records: 21.81 ms

Get all 1,001 records: 4.67 ms
Get record=42 (direct key): 0.15 ms  -> found
Get label='alpha' (index set): 3.07 ms  -> 333 doc(s)

Update record=42: 0.28 ms
Update many label=beta: 27.93 ms  -> 350 record(s)

Delete record=1 : 0.19 ms
Delete label=gamma: 0.86 ms  -> 318 record(s)
flushdb (drop): 0.38 ms
--- --- FINISHED REDIS --- ---
```

```
Redis Benchmark (10,000 records)

--- CREATE ---
Set 1 record: 0.64 ms

Pipeline set 10,000 records: 163.37 ms

Get all 10,001 records: 74.86 ms
Get record=42 (direct key): 0.43 ms  -> found
Get label='alpha' (index set): 25.12 ms  -> 3260 doc(s)

Update record=42: 1.36 ms
Update many label=beta: 230.61 ms  -> 3358 record(s)

Delete record=1 : 0.33 ms
Delete label=gamma: 8.52 ms  -> 3383 record(s)
flushdb (drop): 2.54 ms
--- --- FINISHED REDIS --- ---
```

```
Redis Benchmark (100,000 records)

--- CREATE ---
Set 1 record: 0.88 ms

Pipeline set 100,000 records: 1643.23 ms

Get all 100,001 records: 453.10 ms
Get record=42 (direct key): 0.59 ms  -> found
Get label='alpha' (index set): 157.48 ms  -> 33546 doc(s)

Update record=42: 0.84 ms
Update many label=beta: 1723.23 ms  -> 33159 record(s)

Delete record=1 : 0.53 ms
Delete label=gamma: 74.93 ms  -> 33296 record(s)
flushdb (drop): 19.93 ms
--- --- FINISHED REDIS --- ---
```



These are the graph representations of the time it takes for each operation in ms.

The plots go by bundles of three starting with Postgres on the left, then Redis and finally Mango.

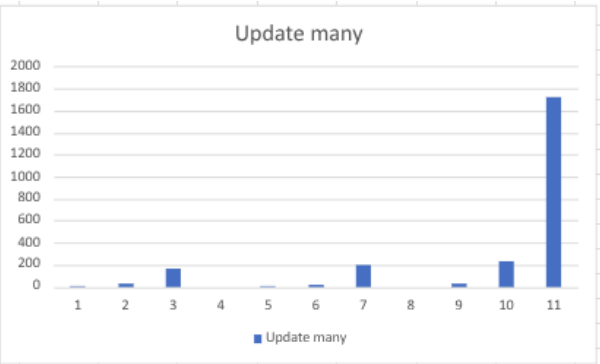
From top to bottom:

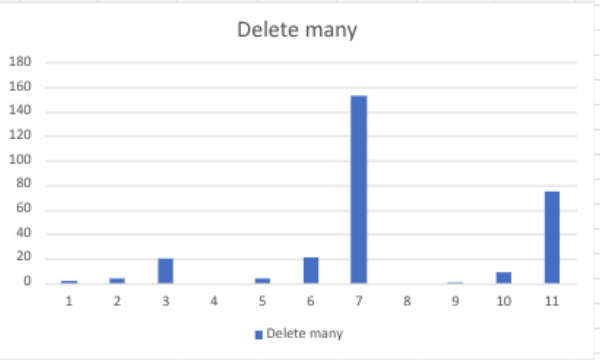
1. Mango seems the fastest with a small growth versus Postgres with random but fast behavior.
2. Postgres and Redis are very close

So for **creation**, Postgres might work the best.

3. Postgres and Redis are very close
4. Postgres
5. Mango
6. Postgres and Redis

So for **reading**, Postgres is the fastest.





From left to right:

1. Too inconclusive to choose
2. Postgres and Redis

For **updating**, Postgres might be better since it's 100'000 time is smaller

3. Mango
4. Postgres
5. Redis

Finally for **deletion**, Postgres is good for anything we want to delete specifically, else Redis is really fast at simply discarding everything.

Fairness

To ensure fairness, I made sure:

- All databases run in docker
- Only one docker was running at a time when testing
- My system's power profile remained the same

There is however, one critical flaw. The Postgres database test is async. This is because I was unable to get psycopg, psycopg2 and pg8000 working for the life of me.

Recommendations

In the overall performance, Postgres seemed to perform best except for the deletion of all the database's content. So the best generalist of all and would be the first choice.

Redis has tended to work less well when dealing with amounts towards the middle, so performs better with large or very small records operations. Would most likely be useful for databases that need to be constantly on while maintaining a steady throughput.

Mango performs relatively well with a very small amount of record operation, so you are geared towards small projects that require speed for small operations.

What worked differently

The aysnc method of Postgres seems to give it quite the advantage, as Redis and Mongodb together are much closer to one another in the overall scores.
Had I known about the async method earlier, I would have tried both sync and async test for Redis and Mongodb to have a more fair comparison with Postgres, but also to see how much it changes from sync.